

DATABASE MANAGEMENT SYSTEM (DBMS)

Complete Question Bank with Full Answers

Pokhara University / NAST Pre-University



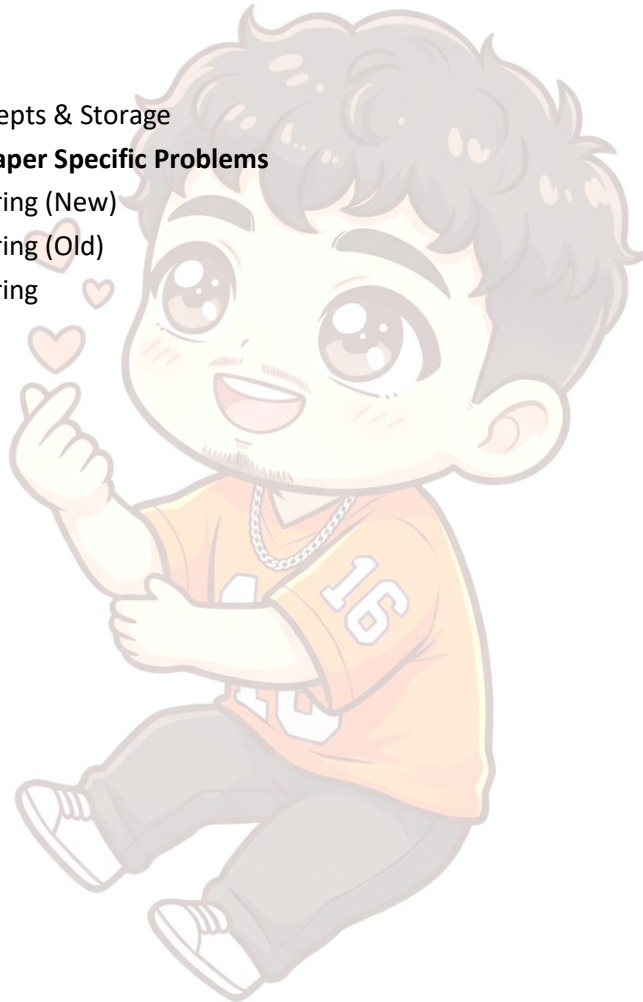
Table of Contents

Part A: Unit-wise Theory Questions

1. Introduction & Architecture
2. ER Diagrams & Relationships
3. Relational Algebra, SQL & Views
4. Constraints & Triggers
5. Normalization & Functional Dependencies
6. Query Processing
7. Transactions & Concurrency Control
8. Backup & Recovery System
9. Database Security
10. Advanced Database Concepts & Storage

Part B: University Question-Paper Specific Problems

- Pokhara University - 2025 Spring (New)
- Pokhara University - 2025 Spring (Old)
- Pokhara University - 2024 Spring
- NAST Pre-University 2025



1. Introduction & Architecture

Q1. Define database and DBMS. What are the advantages of using DBMS over a file management system? [8 Marks]

In Simple English

A database is simply an organised place to keep data — like a digital filing cabinet. A DBMS (Database Management System) is the software that lets people store, find, change, and protect that data easily, instead of digging through hundreds of loose files by hand.

Technical Explanation

Data refers to raw, unprocessed facts and figures (e.g., 21, "Ram", "Kathmandu") that by themselves carry no meaning. **Information** is data that has been processed, organised, or structured to become meaningful and useful for decision-making (e.g., "Ram is 21 years old and lives in Kathmandu").

Database: An organised, persistent collection of logically related data and its description (metadata), designed to be shared by multiple users and applications, and structured to model a real-world enterprise.

DBMS (Database Management System): A software system that enables users to define, create, maintain, and control access to a database. It acts as an interface between the end-user/application program and the physical database, handling storage, retrieval, security, and consistency of data.

Advantages of DBMS over a File Management System

Aspect	File Management System	DBMS
Redundancy	Same data repeated in multiple files	Centralized data — redundancy controlled
Consistency	Updates may not reflect everywhere	Data remains consistent across the system
Data Sharing	Difficult; files are program-specific	Multiple users/applications can share data
Security	Weak, file-level only	Strong, field/row/user-level access control
Integrity	Constraints coded in every program	Constraints enforced centrally by the DBMS
Data Independence	Programs depend heavily on file structure	Applications insulated from storage details
Concurrent Access	Difficult to manage safely	Handled through concurrency control (locking)
Backup & Recovery	Manual, error-prone	Automated backup and recovery mechanisms

In short: a DBMS removes the problems of data redundancy, inconsistency, poor security, and difficult sharing that plague plain file-based systems, while providing a standard query language (SQL), backup/recovery, and multi-user concurrency control.

Q2. Explain the three levels of data abstraction in a DBMS with suitable examples. How does data independence relate to these levels? [8 Marks]

In Simple English

Think of a DBMS like an onion with three layers. The innermost layer is how data is actually stored on disk. The middle layer is the overall logical design that all users share. The outer layer is what each individual user actually sees on their screen. Each layer hides the complexity of the layer below it.

Technical Explanation — The Three Levels (ANSI-SPARC Architecture)

- **Internal (Physical) Level:** Describes how data is physically stored — file organization, indexes, storage blocks, access paths. Example: a Student record stored as a 100-byte block on disk with a B+ tree index on StudentID.
- **Conceptual (Logical) Level:** Describes what data is stored and the relationships among data, independent of physical storage. Example: 'Student(SID, Name, Age, Dept)' and its relationship with 'Course' — this is the community view used by DBAs.
- **External (View) Level:** Describes the part of the database relevant to a particular user/application through views. Example: a Result-Officer's view may only show 'Name, SID, Grade' while hiding 'Salary' or 'Address'.

Three-Level (ANSI-SPARC) Architecture

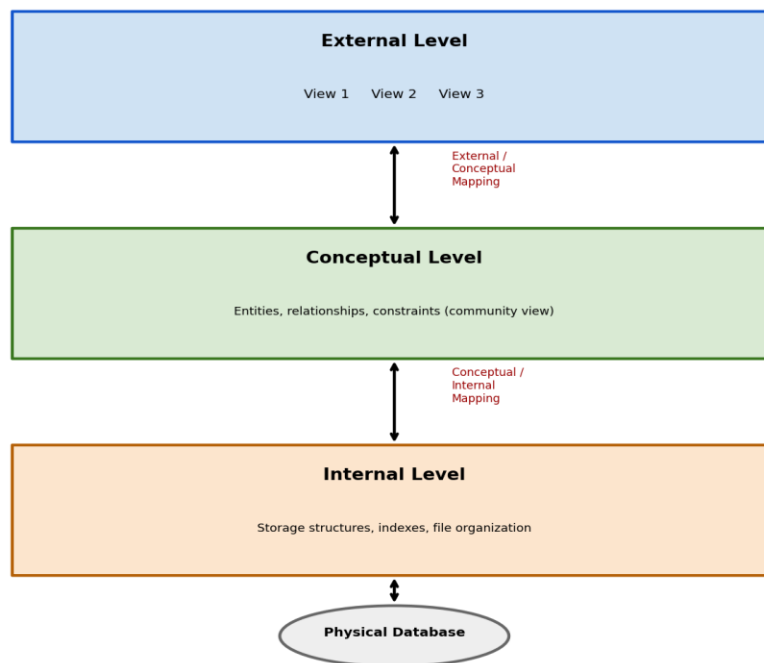


Figure: Three-Level (ANSI-SPARC) Database Architecture

Relation to Data Independence

Data independence is the capacity to change the schema at one level without having to change the schema at the next higher level. The three-level architecture is precisely what makes this possible:

- **Physical Data Independence:** Ability to change the internal/physical schema (e.g., switch file organization, add an index) without altering the conceptual schema or application programs.
- **Logical Data Independence:** Ability to change the conceptual schema (e.g., add a new attribute or entity) without affecting existing external views/application programs. This is harder to achieve than physical independence.



Q3. Define database schema and instances. [8 Marks]

In Simple English

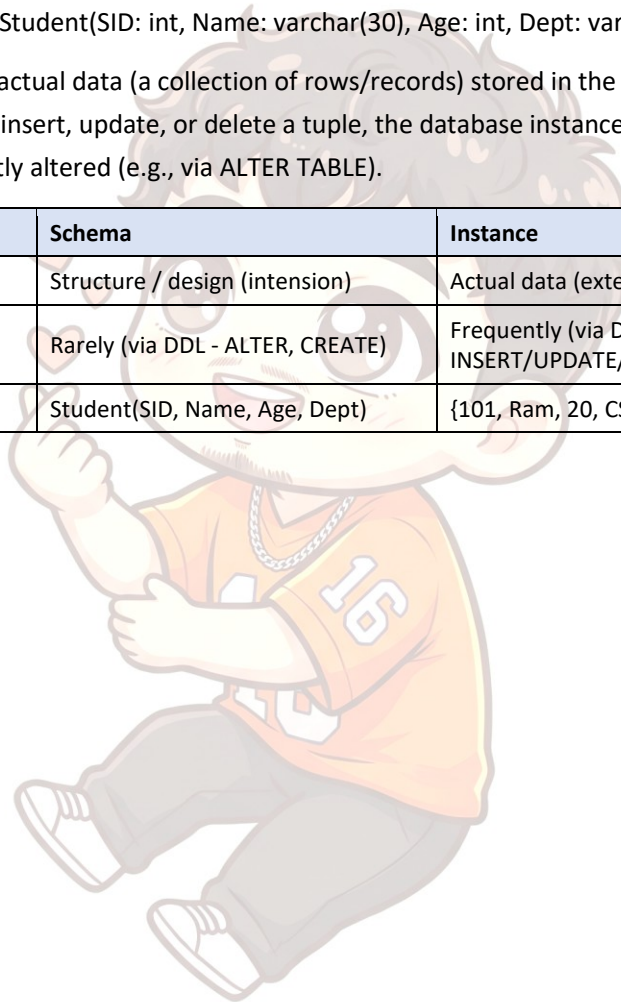
A schema is like a blueprint of a house — it never changes often. An instance is the actual furniture and people inside the house at a given moment — it changes all the time.

Technical Explanation

Database Schema: The overall logical structure/design of the database — the description of tables, attributes, data types, relationships, and constraints — defined at the time of database design. A schema is relatively static and changes infrequently. Example: Student(SID: int, Name: varchar(30), Age: int, Dept: varchar(20)).

Database Instance (State): The actual data (a collection of rows/records) stored in the database at a particular moment in time. Every time we insert, update, or delete a tuple, the database instance changes, but the schema remains the same unless explicitly altered (e.g., via ALTER TABLE).

Feature	Schema	Instance
Nature	Structure / design (intension)	Actual data (extension)
Changes	Rarely (via DDL - ALTER, CREATE)	Frequently (via DML - INSERT/UPDATE/DELETE)
Example	Student(SID, Name, Age, Dept)	{101, Ram, 20, CS}, {102, Sita, 21, IT}



Q4. Explain the two-tier and three-tier database architecture with diagrams. [8 Marks]

In Simple English

Two-tier is like calling a shop directly — your computer (client) talks straight to the database. Three-tier adds a middleman (an application server) between you and the database, so business rules, security, and workload are handled separately from the raw data.

Two-Tier Architecture

In a two-tier architecture, the client application directly communicates with the database server, typically using an API such as ODBC/JDBC. The client handles the presentation and business logic, while the server only manages data storage and retrieval.

Three-Tier Architecture

In a three-tier architecture, an additional Application/Business Logic layer sits between the client and the database server. The client only handles presentation (the user interface), the middle tier processes business rules and application logic, and the database server manages data. This is the standard architecture for modern web applications.

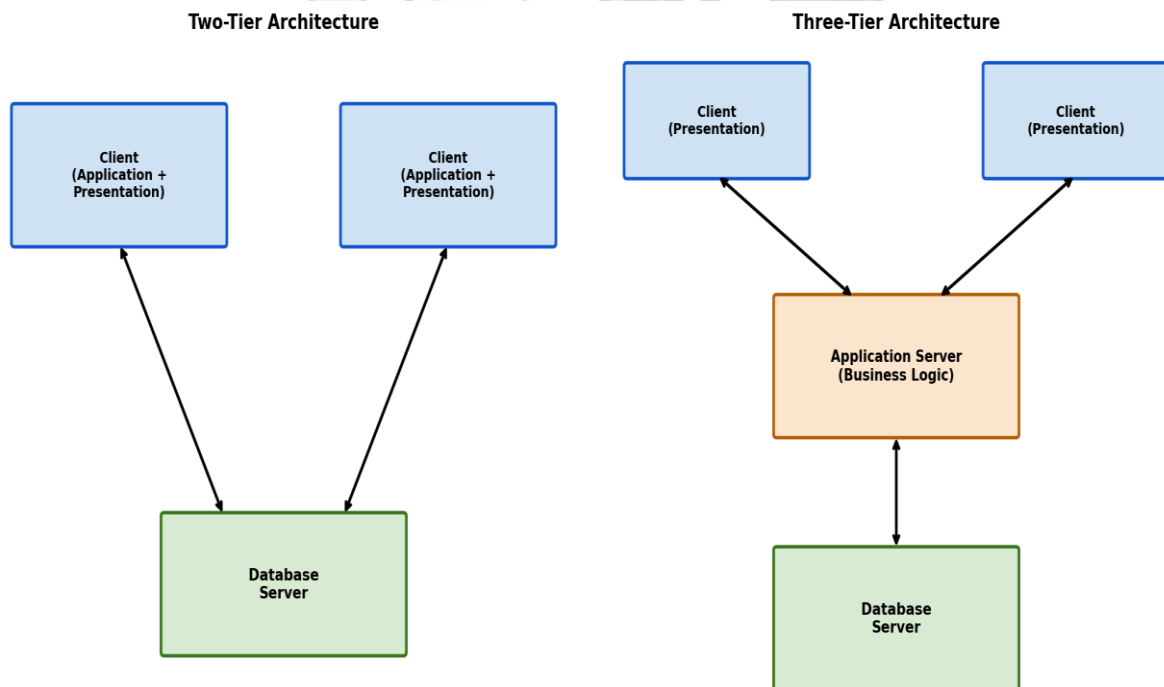


Figure: Two-Tier vs Three-Tier Database Architecture

Feature	Two-Tier	Three-Tier
Layers	Client + Database Server	Client + Application Server + Database Server
Logic Location	On the client	On the middle (application) server
Scalability	Limited	High — layers scale independently
Maintenance	Harder (logic scattered on clients)	Easier — logic centralized on server
Example	Desktop app with direct DB connection	Web application (Browser–App Server–DB)



Q5. What is a data model? Explain the components of the ER model with an example. [8

Marks]

In Simple English

A data model is a set of concepts and rules used to describe the structure of data — like a grammar for describing 'things' and how they relate. The ER (Entity-Relationship) model is one popular data model that describes data using entities, their attributes, and the relationships between them.

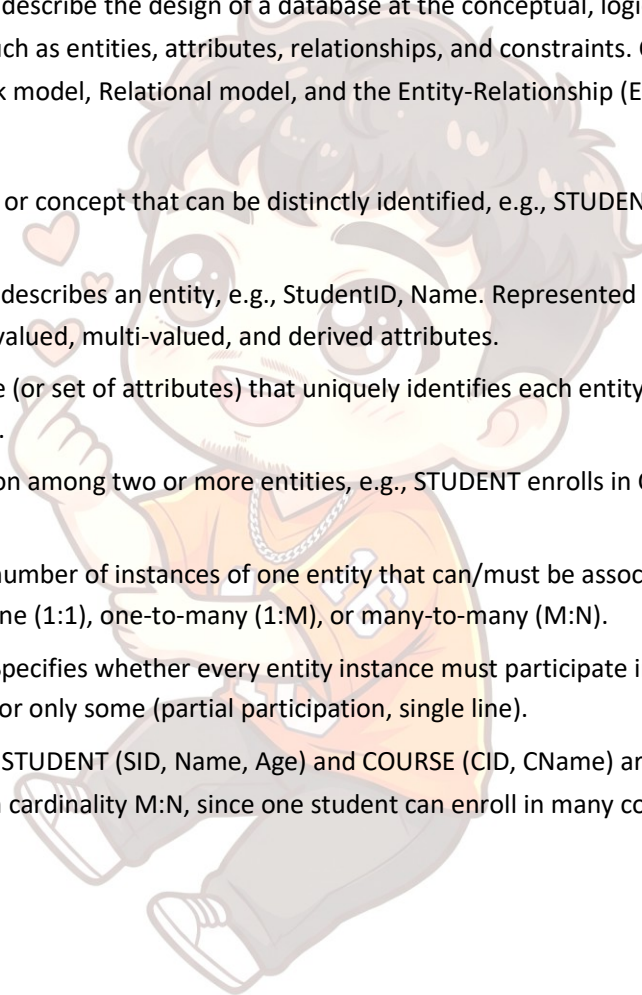
Technical Explanation

A data model provides a way to describe the design of a database at the conceptual, logical, and physical level using a collection of conceptual tools such as entities, attributes, relationships, and constraints. Common data models include the Hierarchical model, Network model, Relational model, and the Entity-Relationship (ER) model.

Components of the ER Model

- **Entity:** A real-world object or concept that can be distinctly identified, e.g., STUDENT, COURSE. Represented as a rectangle.
- **Attribute:** A property that describes an entity, e.g., StudentID, Name. Represented as an oval. Types include simple, composite, single-valued, multi-valued, and derived attributes.
- **Key Attribute:** An attribute (or set of attributes) that uniquely identifies each entity instance, e.g., StudentID. Underlined in ER diagrams.
- **Relationship:** An association among two or more entities, e.g., STUDENT enrolls in COURSE. Represented as a diamond.
- **Cardinality:** Specifies the number of instances of one entity that can/must be associated with instances of another entity — one-to-one (1:1), one-to-many (1:M), or many-to-many (M:N).
- **Participation Constraint:** Specifies whether every entity instance must participate in the relationship (total participation, double line) or only some (partial participation, single line).

Example: In a College database, STUDENT (SID, Name, Age) and COURSE (CID, CName) are entities. ENROLLS_IN is a relationship between them with cardinality M:N, since one student can enroll in many courses and one course can have many students.



2. ER Diagrams & Relationships

Q1. Draw an ER diagram for a hospital system (Patient, Doctor, Appointment, Room) with appropriate relationships and cardinalities. [8 Marks]

In Simple English

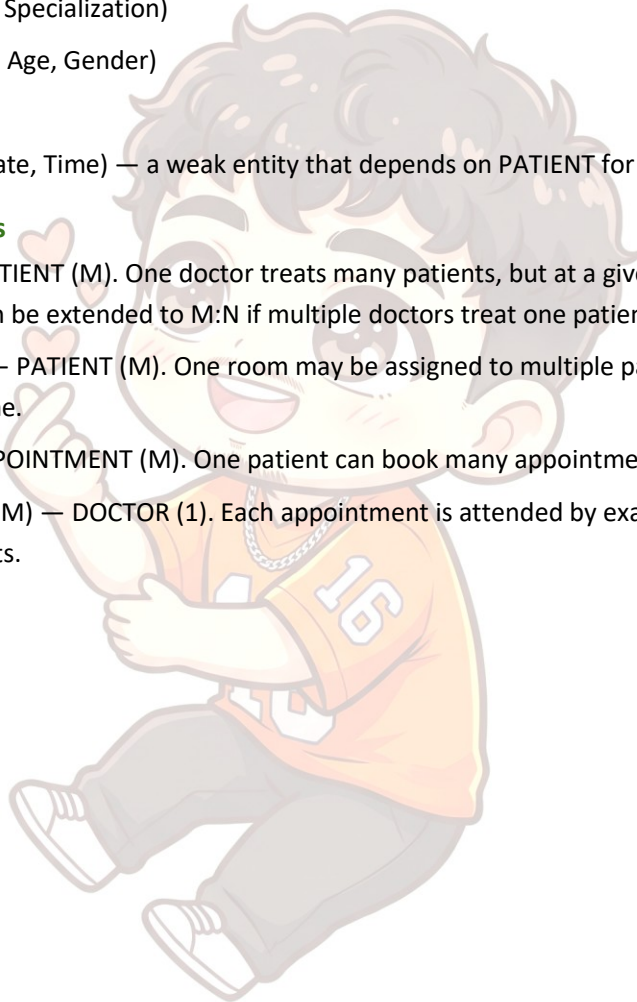
This diagram shows how patients, doctors, rooms, and appointments in a hospital are connected — which doctor treats which patient, which room a patient is assigned, and how appointments are booked.

Entities and Attributes

- **DOCTOR** (DoctorID, Name, Specialization)
- **PATIENT** (PatientID, Name, Age, Gender)
- **ROOM** (RoomNo, Type)
- **APPOINTMENT** (ApptID, Date, Time) — a weak entity that depends on PATIENT for its existence

Relationships and Cardinalities

- **TREATS:** DOCTOR (1) — PATIENT (M). One doctor treats many patients, but at a given time each patient's case is handled by one doctor (can be extended to M:N if multiple doctors treat one patient).
- **ASSIGNED TO:** ROOM (1) — PATIENT (M). One room may be assigned to multiple patients over time; a patient is assigned one room at a time.
- **BOOKS:** PATIENT (1) — APPOINTMENT (M). One patient can book many appointments.
- **ATTENDS:** APPOINTMENT (M) — DOCTOR (1). Each appointment is attended by exactly one doctor; a doctor attends many appointments.



ER Diagram: Hospital Management System

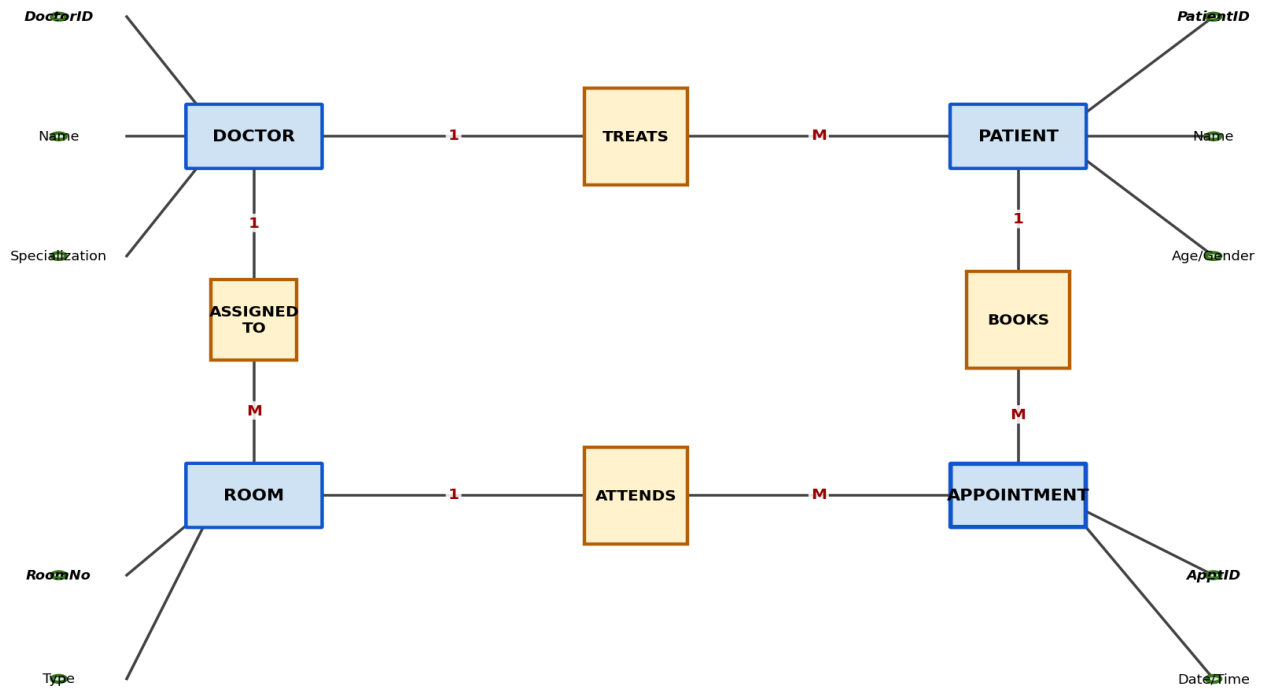
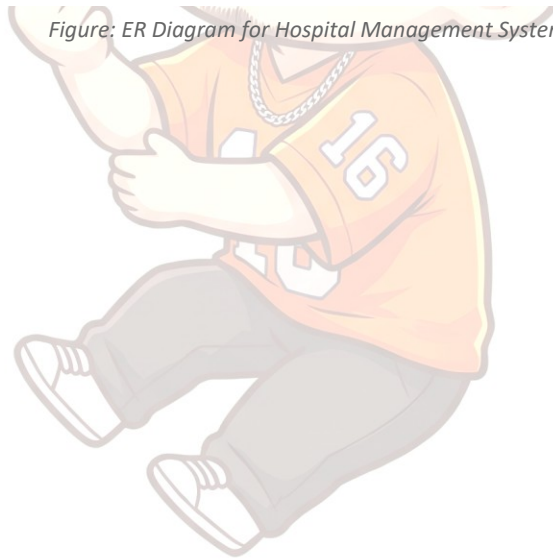


Figure: ER Diagram for Hospital Management System



Q2. Draw an ER diagram for an "online food ordering system" using extended features like generalization and specialization. [8 Marks]

In Simple English

This is the same idea as a normal ER diagram, but we add a special feature: PAYMENT is a general concept that can be split into more specific types — Card Payment, Cash Payment, or Wallet Payment. This splitting is called generalization/specialization.

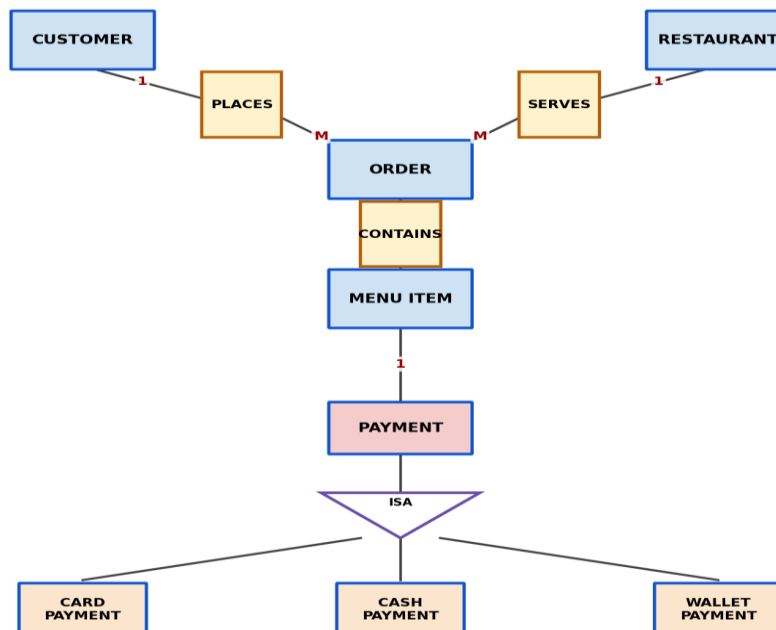
Entities and Relationships

- **CUSTOMER** (CustomerID, Name, Address, Phone) —PLACES→ ORDER (1:M)
- **RESTAURANT** (RestID, Name, Location) —SERVES→ ORDER (1:M)
- **ORDER** (OrderID, OrderDate, Status) —CONTAINS→ MENU ITEM (M:N via OrderDetails)
- **MENU ITEM** (ItemID, Name, Price)
- **PAYMENT** (PaymentID, Amount, Date) — generalized entity, related to ORDER (1:1)

Extended (EER) Feature: Generalization / Specialization

PAYMENT is specialized into three disjoint sub-types: CARD PAYMENT (CardNo, ExpiryDate), CASH PAYMENT (ReceivedBy), and WALLET PAYMENT (WalletProvider, TransactionID). This is shown using an ISA (triangle) symbol connecting the superclass PAYMENT to its subclasses. Since every payment must belong to exactly one sub-type, this is total and disjoint specialization.

EER Diagram: Online Food Ordering System



Generalization/Specialization: PAYMENT is generalized into CARD, CASH and WALLET payment sub-types (disjoint, total participation)

Figure: EER Diagram for Online Food Ordering System (with Generalization/Specialization)

Q3. Design an ER diagram for a Library Management System (Members, Books, Borrow/Return tracking). [8 Marks]

In Simple English

This diagram tracks which library member has borrowed which book, and when it was issued and returned.

Entities and Attributes

- **MEMBER** (MemberID, Name, Address, Phone)
- **BOOK** (ISBN, Title, Author, Publisher)

Relationship

BORROWS/RETURNS is a many-to-many (M:N) relationship between MEMBER and BOOK, because one member can borrow many books over time and one book (across different copies/times) can be borrowed by many members.

Since the relationship itself carries data, it has its own descriptive attributes:

- IssueDate — the date the book was issued to the member
- DueDate — the date by which the book must be returned
- ReturnDate — the actual date of return (null until returned)

In relational implementation, this M:N relationship is converted into a separate 'Borrow' table with MemberID and ISBN as a composite foreign key, plus IssueDate, DueDate, and ReturnDate as its own attributes.

ER Diagram: Library Management System

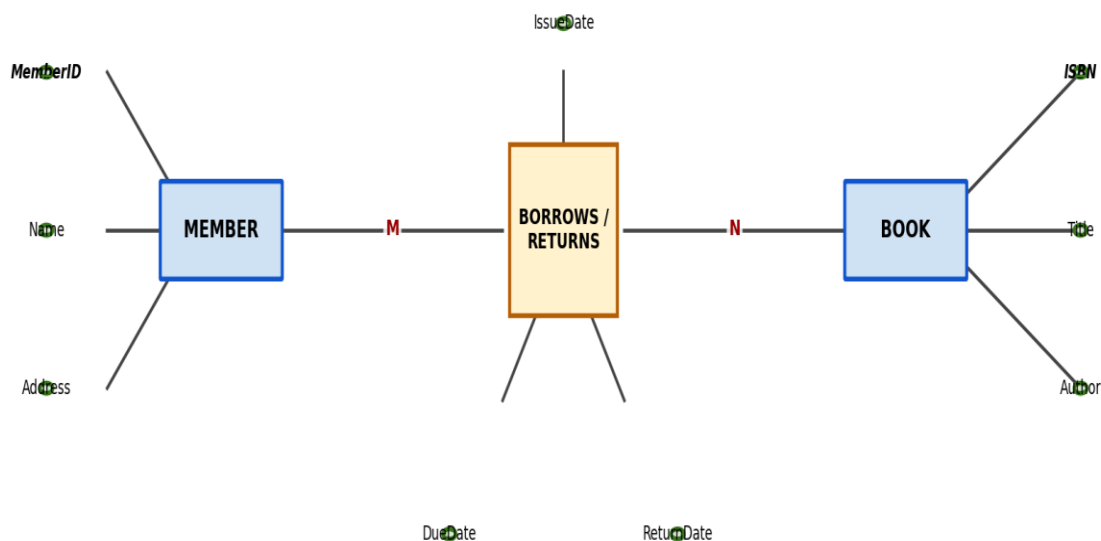


Figure: ER Diagram for Library Management System

Q4. Write a short note on: Generalization VS Aggregation. [8 Marks]

In Simple English

Generalization groups similar 'types' of things into one bigger category (like Car and Truck both being a Vehicle).

Aggregation treats a whole relationship as if it were a single entity, so it can be connected to something else.

Technical Explanation

Generalization: The process of defining a higher-level (super-class) entity from two or more lower-level (sub-class) entities that share common attributes, following a bottom-up approach. It is represented using an ISA relationship. Example: CAR and TRUCK are generalized into VEHICLE.

Aggregation: A modelling concept used to express a relationship between a relationship (or a set of entity-relationship-entity) and another entity, when a relationship itself needs to be related to a further entity. It abstracts a relationship as a higher-level entity.

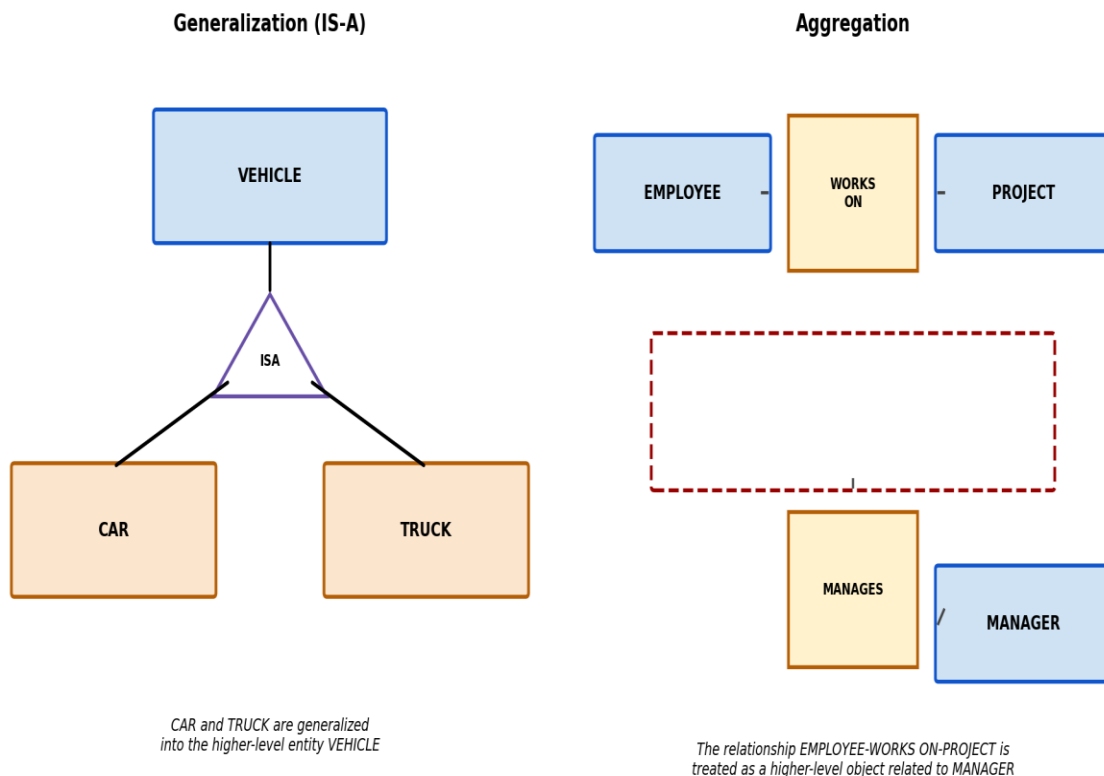


Figure: Generalization (IS-A) vs Aggregation

Aspect	Generalization	Aggregation
Purpose	Combines similar entities into one super-type	Treats a relationship as an entity
Direction	Bottom-up (specific → general)	Groups relationship + entities → higher unit
Notation	ISA / triangle symbol	Relationship enclosed in a box/diamond treated as an entity
Example	CAR, TRUCK → VEHICLE	(EMPLOYEE-WORKS_ON-PROJECT) → MANAGES → MANAGER



3. Relational Algebra, SQL & Views

Q1. Write relational algebra expressions for given schemas (Student/Course, Employee/Project, etc.). [8 Marks]

In Simple English

Relational algebra is a set of basic 'building block' operations (like select, project, join) that we combine step-by-step to pull out exactly the data we want from tables — it is the theoretical foundation behind SQL.

Technical Explanation — Basic Operators

Operator	Symbol	Purpose
Selection	σ (sigma)	Selects rows/tuples that satisfy a given condition
Projection	π (pi)	Selects specific columns/attributes, removing duplicates
Union	$\cup$	Combines tuples from two union-compatible relations
Set Difference	$-$	Tuples in R1 but not in R2
Cartesian Product	$\times$	Combines every tuple of R1 with every tuple of R2
Rename	ρ (rho)	Renames a relation or its attributes
Join	$\bowtie$	Combines related tuples from two relations based on a condition

Example — Student(SID, Name, Age, Dept), Course(CID, CName, Credits), Enroll(SID, CID, Grade)

(i) Names of students enrolled in "Database Systems":

```
 $\pi$  Name (  $\sigma$  CName='Database Systems' ( Student  $\bowtie$  Enroll  $\bowtie$  Course ) )
```

(ii) Students not enrolled in any course:

```
 $\pi$  SID,Name ( Student ) -  $\pi$  SID,Name ( Student  $\bowtie$  Enroll )
```

(iii) Courses with more than 3 credits:

```
 $\pi$  CID,CName (  $\sigma$  Credits > 3 ( Course ) )
```

(iv) Total number of students in each department (using the aggregate/grouping extension of relational algebra):

```
Dept  $\bowtie$  COUNT(SID) as Total ( Student )
```

Similarly, for Employee(EmpID, Name, Position, Age) and Project(PID, Name, Duration, Budget) related through Works(EmpID, PID, Location, Salary), expressions such as finding managers above 30 years, projects running under 45 days, or updating/deleting tuples follow the same pattern of σ (selection), π (projection), and $\bowtie$ (join), while modification is expressed conceptually since pure relational algebra is a query language, not an update language (updates are shown symbolically as reassignment: $R \leftarrow R - \{\text{tuple}\} \cup \{\text{new tuple}\}$).

Q2. Write SQL queries/statements for given database schemas. [8 Marks]

In Simple English

SQL (Structured Query Language) is the standard language used to actually talk to a real database — to create tables, insert data, and ask questions like 'show me all employees earning more than 50,000'.

Technical Explanation — Categories of SQL Commands

Category	Full Form	Example Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT

Illustrative Example — Employees(EmpID, Name, DeptID, Salary, JoinDate)

```
-- Create table with constraints
CREATE TABLE Employees (
    EmpID      INT PRIMARY KEY,
    Name       VARCHAR(50) NOT NULL,
    DeptID     INT,
    Salary     DECIMAL(10,2) CHECK (Salary > 0),
    JoinDate   DATE,
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

-- Retrieve data
SELECT Name, Salary FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
WHERE d.DeptName = 'IT';

-- Update data
UPDATE Employees SET Salary = Salary * 1.10
WHERE DeptID = (SELECT DeptID FROM Departments WHERE DeptName='IT');

-- Delete data
DELETE FROM Projects WHERE Budget < 40000;
```

Detailed, schema-specific query answers for every past-paper question are worked out in Part B of this document, matching each university's exact table names.

Q3. What is a view? Explain the applications/importance of a view in DBMS with its syntax.

[8 Marks]

In Simple English

A view is like a saved, virtual table created from a query — it does not store data itself, but shows selected data from one or more real tables whenever you look at it.

Technical Explanation

A view is a virtual/derived relation whose contents are defined by a stored SQL query. It does not physically store data (unless materialized); instead it is computed dynamically whenever it is referenced.

Syntax

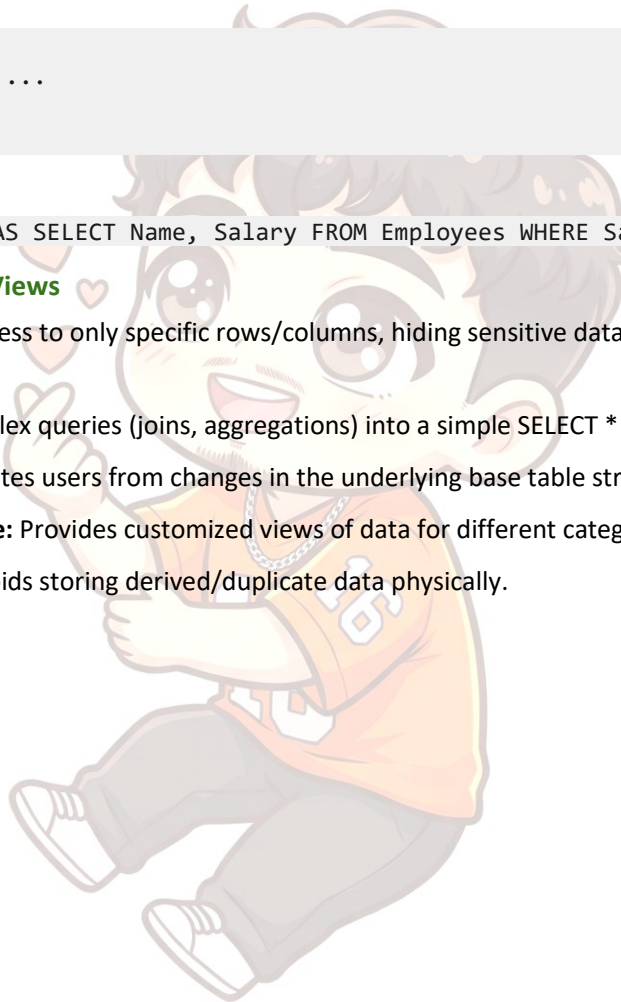
```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Example:

```
CREATE VIEW HighEarnings AS SELECT Name, Salary FROM Employees WHERE Salary > 100000;
```

Applications / Importance of Views

- **Security:** Restricts user access to only specific rows/columns, hiding sensitive data (e.g., hiding Salary from junior staff).
- **Simplicity:** Simplifies complex queries (joins, aggregations) into a simple SELECT * FROM view.
- **Data Independence:** Insulates users from changes in the underlying base table structure.
- **Logical Data Independence:** Provides customized views of data for different categories of users.
- **Reduced Redundancy:** Avoids storing derived/duplicate data physically.



Q4. What is a JOIN? Differentiate between inner join and outer join. [8 Marks]

In Simple English

A join combines data from two or more tables that are related to each other, usually through a common column (like a foreign key).

Technical Explanation

A JOIN operation combines rows from two or more tables based on a related column, allowing us to retrieve data that is spread across multiple normalized tables.

Types of Joins

- **INNER JOIN:** Returns only the rows that have matching values in both tables.
- **LEFT (OUTER) JOIN:** Returns all rows from the left table, plus matched rows from the right table (unmatched right-side columns are NULL).
- **RIGHT (OUTER) JOIN:** Returns all rows from the right table, plus matched rows from the left table.
- **FULL (OUTER) JOIN:** Returns all rows from both tables, matching where possible and filling NULL where there is no match.

```
-- Inner Join
SELECT e.Name, d.DeptName FROM Employees e
INNER JOIN Departments d ON e.DeptID = d.DeptID;

-- Left Outer Join
SELECT e.Name, d.DeptName FROM Employees e
LEFT JOIN Departments d ON e.DeptID = d.DeptID;
```

Feature	Inner Join	Outer Join
Rows returned	Only matching rows from both tables	All rows from one/both tables even if unmatched
Unmatched Data	Discarded	Kept, shown as NULL
Types	Single type	LEFT, RIGHT, FULL
Use Case	When only related data matters	When you need to preserve all records, e.g., all customers even those with no orders

Q5. What are Set Operations in DBMS? Explain UNION and INTERSECTION. [8 Marks]

In Simple English

Because tables are basically sets of rows, we can combine two similar tables using set operations, just like combining two sets in mathematics.

Technical Explanation

SQL supports mathematical set operations on the result of two or more SELECT queries, provided the queries are union-compatible (same number of columns with compatible data types).

- **UNION:** Combines the results of two queries and removes duplicate rows. UNION ALL keeps duplicates.
- **INTERSECT (Intersection):** Returns only the rows that appear in both result sets.
- **MINUS / EXCEPT (Difference):** Returns rows present in the first query but not in the second.

```
-- UNION: all students who are either in CS dept or enrolled in DBMS course
SELECT SID FROM Student WHERE Dept='CS'
UNION
SELECT SID FROM Enroll WHERE CID='C101';
```

```
-- INTERSECT: students who are in CS dept AND enrolled in DBMS course
SELECT SID FROM Student WHERE Dept='CS'
INTERSECT
SELECT SID FROM Enroll WHERE CID='C101';
```



Q6. Write a short note on QBE (Query By Example). [8 Marks]

In Simple English

QBE lets you ask a database a question by filling an example table on screen instead of typing a text-based query — you basically show the system 'an example of what you want'.

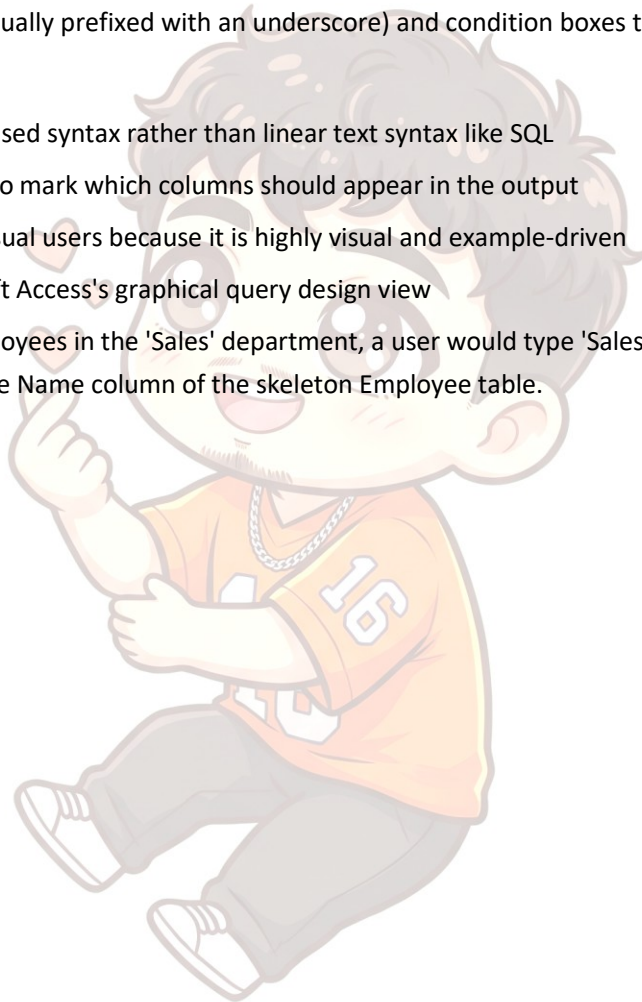
Technical Explanation

Query By Example (QBE) is a visual/graphical query language developed by IBM that allows users to express queries by filling in blank tables (skeleton tables) shown on screen, rather than writing textual SQL statements. It uses example elements (variables, usually prefixed with an underscore) and condition boxes to specify search criteria.

Key Features

- Two-dimensional, table-based syntax rather than linear text syntax like SQL
- Uses the letter 'P.' (Print) to mark which columns should appear in the output
- Ideal for non-technical/casual users because it is highly visual and example-driven
- Was the basis for Microsoft Access's graphical query design view

Example: To find names of employees in the 'Sales' department, a user would type 'Sales' under the Department column and 'P._Name' under the Name column of the skeleton Employee table.



4. Constraints & Triggers

Q1. What are integrity constraints, and why do we need them? Explain domain, entity, and referential integrity with examples. [8 Marks]

In Simple English

Integrity constraints are simply rules that keep the data in a database correct and sensible — like making sure age can't be negative, or that every order refers to a customer who actually exists.

Technical Explanation

Integrity constraints are rules that a database must always satisfy, ensuring the accuracy, consistency, and reliability of the data stored. They are enforced automatically by the DBMS during INSERT, UPDATE, and DELETE operations, preventing accidental corruption of data.

Why We Need Them

- To prevent invalid or meaningless data from entering the database
- To maintain consistency between related tables
- To enforce business rules automatically, without relying on application code
- To protect against accidental data loss caused by careless updates or deletions

Types of Integrity Constraints

1. Domain Integrity: Ensures that every value in a column falls within its defined domain (data type, range, format). Example: Age INT CHECK (Age > 0) — ensures Age can never be negative.

2. Entity Integrity: Ensures that the primary key of a table is unique and not NULL, so that every row/entity can be uniquely identified. Example: StudentID INT PRIMARY KEY — no two students can have the same ID, and it cannot be left blank.

3. Referential Integrity: Ensures that a foreign key value in one table must either match a primary key value in the referenced table or be NULL — preventing 'orphan' records. Example: Enroll.SID must exist in Student.SID; you cannot enroll a student who doesn't exist.

```
CREATE TABLE Enroll (  
    SID INT,  
    CID INT,  
    Grade CHAR(2),  
    FOREIGN KEY (SID) REFERENCES Student(SID),  
    FOREIGN KEY (CID) REFERENCES Course(CID)  
);
```


Q2. Explain trigger and assertion with examples. [8 Marks]

In Simple English

A trigger is a rule that automatically runs a piece of code whenever something happens in the database (like an insert or update). An assertion is a rule that must always hold true across the whole database — the system checks it constantly, not just at one moment.

Technical Explanation — Trigger

A trigger is a stored procedure that automatically executes (fires) in response to a specific event (INSERT, UPDATE, DELETE) on a specified table. Triggers help enforce complex business rules that cannot be expressed with simple constraints alone.

```
CREATE TRIGGER trg_CheckSalary
BEFORE INSERT ON Employees
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SET NEW.Salary = 0;
    END IF;
END;
```

Technical Explanation — Assertion

An assertion is a general-purpose SQL constraint that specifies a condition that must always be satisfied by the database, independent of any single table or event. Unlike triggers, assertions are not tied to a triggering event — the DBMS checks them whenever any relevant data changes.

```
CREATE ASSERTION MinRoomCapacity
CHECK (
    NOT EXISTS (
        SELECT * FROM Room WHERE Capacity < 1
    )
);
```

Feature	Trigger	Assertion
Activation	Fires on a specific event (INSERT/UPDATE/DELETE)	Checked whenever any relevant table changes
Scope	Usually one table	Can span multiple tables
Action	Can perform actions (modify data, raise error)	Only checks a condition; does not perform actions
Support	Widely supported (MySQL, Oracle, SQL Server)	Rarely implemented fully in commercial DBMS

5. Normalization & Functional Dependencies

Q1. Why do we need normalization? Differentiate 2NF and 3NF with appropriate examples.
[8 Marks]

In Simple English

Normalization is the process of organizing table columns so that the same fact isn't repeated in many places, which prevents confusing or contradictory data when we insert, update, or delete records.

Why We Need Normalization

- To eliminate data redundancy (avoid storing the same fact multiple times)
- To avoid insertion, update, and deletion anomalies
- To ensure that functional dependencies make logical sense (attributes depend only on the key)
- To make the database easier to maintain and modify

Second Normal Form (2NF)

A relation is in 2NF if it is already in 1NF (atomic values, no repeating groups) and every non-key attribute is fully functionally dependent on the entire primary key — i.e., there is no partial dependency on part of a composite key.

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and has no transitive dependency — i.e., no non-key attribute depends on another non-key attribute.

Aspect	2NF	3NF
Requires	1NF + no partial dependency	2NF + no transitive dependency
Problem removed	Non-key attributes depending on part of a composite key	Non-key attributes depending on other non-key attributes
Example issue	In Enroll(SID, CID, SName, Grade): SName depends only on SID, not on the whole key (SID,CID) → violates 2NF	In Employee(EmpID, DeptID, DeptName): DeptName depends on DeptID, not directly on EmpID → violates 3NF
Fix	Split SName into a separate Student table	Split DeptName into a separate Department table

Q2. Convert a given 2NF relation into 3NF. Check if any insertion, updation, or deletion anomaly occurs. [8 Marks]

In Simple English

Here we take a table that is already free of partial dependency but still has one column depending on another non-key column, and split it further so every column depends only on the key.

Example: 2NF Relation

Consider: Employee(Name, Address, Phone, Salary, Post) — with Name as the primary key. Here, Salary depends on Post (every employee with the same Post earns the same Salary), not directly on Name. This is a transitive dependency: Name → Post → Salary.

Name	Address	Phone	Salary	Post
Ram	Pokhara	9800...	40000	Manager
Sita	Kathmandu	9810...	40000	Manager
Hari	Butwal	9820...	25000	Clerk

Anomalies Present (Before Normalization to 3NF)

- **Insertion Anomaly:** We cannot add a new Post (e.g., 'Accountant') and its salary until at least one employee holding that post is inserted.
- **Update Anomaly:** If the salary for 'Manager' changes, we must update it in every row where Post='Manager' — missing even one row causes inconsistent data.
- **Deletion Anomaly:** If Hari (the only Clerk) is deleted, we lose all information that Clerks earn 25000 — the fact disappears along with the employee.

Decomposition into 3NF

Split into two relations to remove the transitive dependency Name → Post → Salary:

```
Employee (Name, Address, Phone, Post)    -- Name is Primary Key
PostSalary (Post, Salary)                -- Post is Primary Key
```

Now Salary depends directly and only on Post (its own key) in the PostSalary table, and every attribute in Employee depends only on Name. All three anomalies described above are eliminated: a new post's salary can be added independently, salary updates happen in a single place, and deleting an employee no longer erases post/salary information.

Q3. What is denormalization? When and why is it used in database design? [8 Marks]

In Simple English

Denormalization is the opposite of normalization — deliberately allowing some repeated data back into tables to make reading data faster, at the cost of some redundancy.

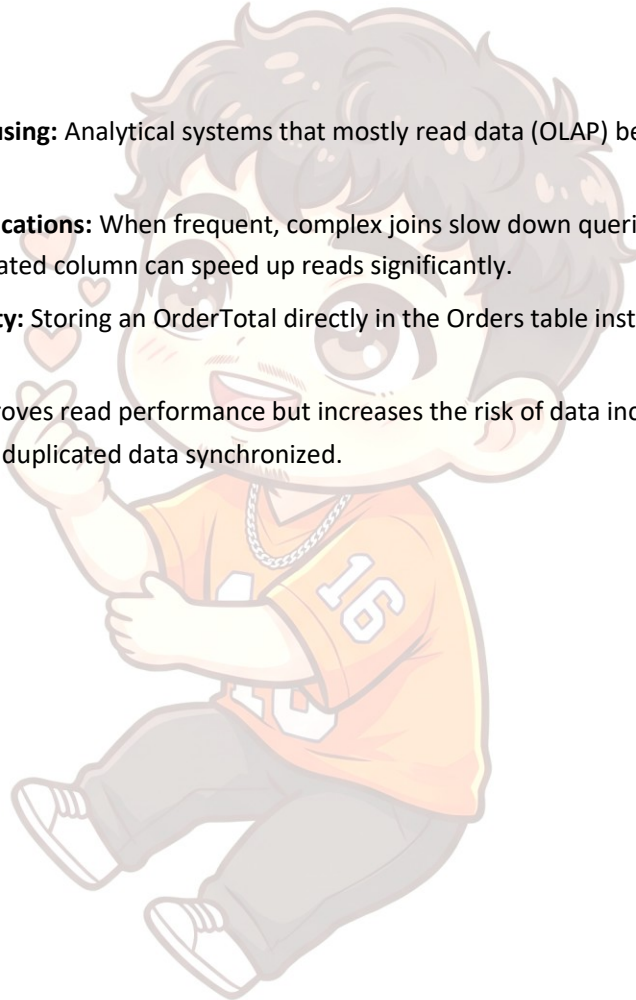
Technical Explanation

Denormalization is the process of intentionally introducing redundancy into a normalized database by combining tables or adding redundant columns, in order to improve read/query performance by reducing the number of joins required.

When and Why It Is Used

- **Reporting / Data Warehousing:** Analytical systems that mostly read data (OLAP) benefit from fewer joins, since they rarely update data.
- **Performance-Critical Applications:** When frequent, complex joins slow down queries in high-traffic applications, storing a computed/duplicated column can speed up reads significantly.
- **Reducing Query Complexity:** Storing an OrderTotal directly in the Orders table instead of recalculating it every time from OrderDetails.

Trade-off: Denormalization improves read performance but increases the risk of data inconsistency and requires more careful application logic to keep duplicated data synchronized.



Q4. What is functional dependency? Explain the types of functional dependency. [8 Marks]

In Simple English

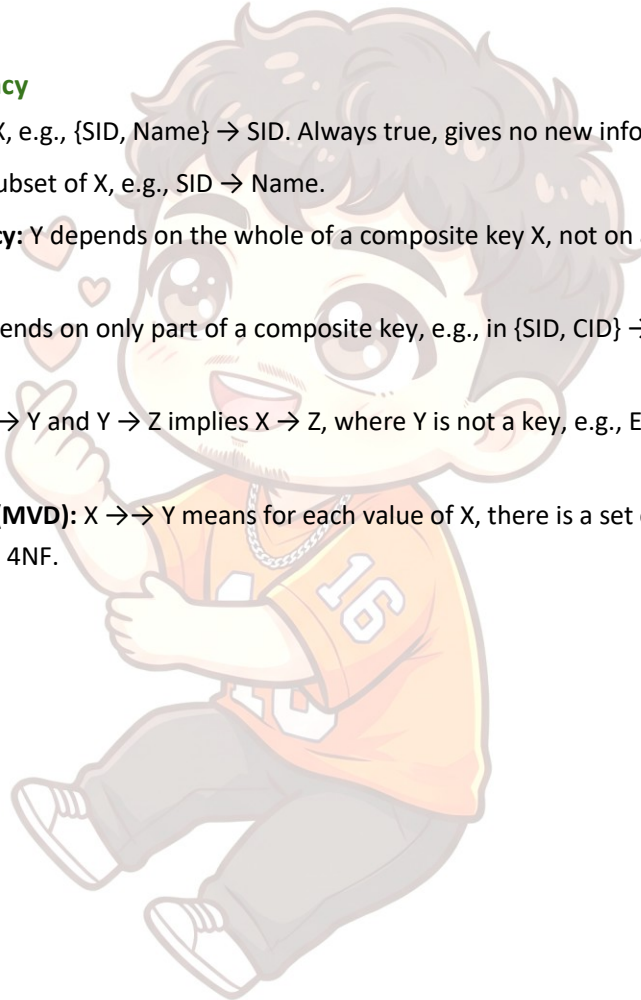
A functional dependency is just a fancy way of saying: if you know one piece of information, you automatically know another. For example, if you know a student's ID, you automatically know their name.

Technical Explanation

A functional dependency (FD), written $X \rightarrow Y$, means that the value of attribute set X uniquely determines the value of attribute set Y . That is, for any two tuples having the same value of X , they must also have the same value of Y . X is called the determinant.

Types of Functional Dependency

- **Trivial FD:** Y is a subset of X , e.g., $\{SID, Name\} \rightarrow SID$. Always true, gives no new information.
- **Non-Trivial FD:** Y is not a subset of X , e.g., $SID \rightarrow Name$.
- **Full Functional Dependency:** Y depends on the whole of a composite key X , not on any proper subset of it, e.g., $\{SID, CID\} \rightarrow Grade$.
- **Partial Dependency:** Y depends on only part of a composite key, e.g., in $\{SID, CID\} \rightarrow SName$, $SName$ depends only on SID .
- **Transitive Dependency:** $X \rightarrow Y$ and $Y \rightarrow Z$ implies $X \rightarrow Z$, where Y is not a key, e.g., $EmpID \rightarrow DeptID \rightarrow DeptName$.
- **Multivalued Dependency (MVD):** $X \twoheadrightarrow Y$ means for each value of X , there is a set of values of Y , independent of other attributes; used in 4NF.



6. Query Processing

Q1. What is query processing/optimization? Explain the need and steps involved in query processing. [8 Marks]

In Simple English

When you type an SQL query, the DBMS doesn't just run it blindly — it first figures out the smartest, fastest way to actually fetch the data. That whole planning-and-executing process is called query processing, and picking the cheapest plan is called query optimization.

Technical Explanation

Query processing is the set of activities involved in translating a high-level query (SQL) into low-level operations that can be executed on the physical database, and then actually executing them to produce the result. Query optimization is the process of selecting the most efficient execution plan (in terms of cost — disk I/O, CPU time, memory) among many equivalent plans that produce the same result.

Why We Need It

- The same query can be executed in many different ways (different join orders, algorithms, index usage) — some far more efficient than others
- Without optimization, even a simple query on large tables could take an unacceptably long time
- It reduces resource usage (disk I/O, memory, CPU) and improves response time for users

Steps in Query Processing

1. **Parsing and Translation:** The SQL query is parsed for syntax and semantic correctness, then translated into an internal representation such as a relational algebra expression or a query tree/graph.
2. **Query Optimization:** The optimizer generates multiple equivalent execution plans (e.g., different join orders, choice of index vs full scan) and estimates the cost of each using statistics (table size, index availability, selectivity), then selects the plan with the lowest estimated cost.
3. **Query Code Generation / Compilation:** The chosen low-cost execution plan is compiled into actual code or a sequence of operations that the query-execution engine can run.
4. **Execution:** The query-execution engine runs the generated code against the database and produces the final result, which is returned to the user/application.

Together, these steps ensure that the DBMS finds and executes an efficient strategy for answering each query, rather than simply computing the query in a naive, literal manner.

7. Transactions & Concurrency Control

Q1. What is concurrency control? Explain how two-phase locking (2PL) and timestamp-based protocols are used to control concurrency. [8 Marks]

In Simple English

When many people use a database at the same time, their actions could clash — like two people trying to book the last movie seat simultaneously. Concurrency control is the set of rules that stops such clashes from corrupting the data.

Technical Explanation

Concurrency control refers to the mechanisms used by a DBMS to ensure that concurrent execution of multiple transactions leaves the database in a consistent state, exactly as if the transactions had executed one after another (serially).

Two-Phase Locking (2PL) Protocol

2PL is a lock-based protocol where every transaction acquires and releases locks in two distinct phases:

- **Growing Phase:** The transaction may acquire (obtain) locks but cannot release any lock.
- **Shrinking Phase:** The transaction may release locks but cannot acquire any new lock.

Once a transaction releases its first lock, it enters the shrinking phase and cannot request further locks. 2PL guarantees conflict serializability. A stricter variant, Strict 2PL, holds all exclusive locks until the transaction commits or aborts, avoiding cascading rollbacks.

Timestamp-Based Protocol

Each transaction T_i is assigned a unique timestamp $TS(T_i)$ when it enters the system, based on the system clock or a logical counter. The protocol ensures serializability by ordering conflicting operations according to their timestamps: an older transaction is given priority over a younger one. Every data item X maintains a Read-Timestamp(X) and Write-Timestamp(X):

- If T_i wants to read X and $TS(T_i) < \text{Write-Timestamp}(X)$, T_i is rejected and rolled back (it would be reading outdated data out of order)
- If T_i wants to write X and $TS(T_i) < \text{Read-Timestamp}(X)$ or $TS(T_i) < \text{Write-Timestamp}(X)$, T_i is rejected and rolled back
- Otherwise the operation proceeds and the relevant timestamp on X is updated

Unlike 2PL, the timestamp-based protocol avoids deadlocks entirely, since transactions never wait for locks — conflicting transactions are simply rolled back and restarted with a new timestamp.

Q2. Define Serializability. Explain conflict serializability and view serializability with examples. [8 Marks]

In Simple English

Serializability just means: even though many transactions run at the same time (interleaved) for speed, the final result must look exactly as if they had run one after another in some order.

Technical Explanation

Serializability is a correctness criterion for concurrent schedules — a schedule is serializable if its effect on the database is equivalent to the effect of some serial (one-at-a-time) execution of the same transactions.

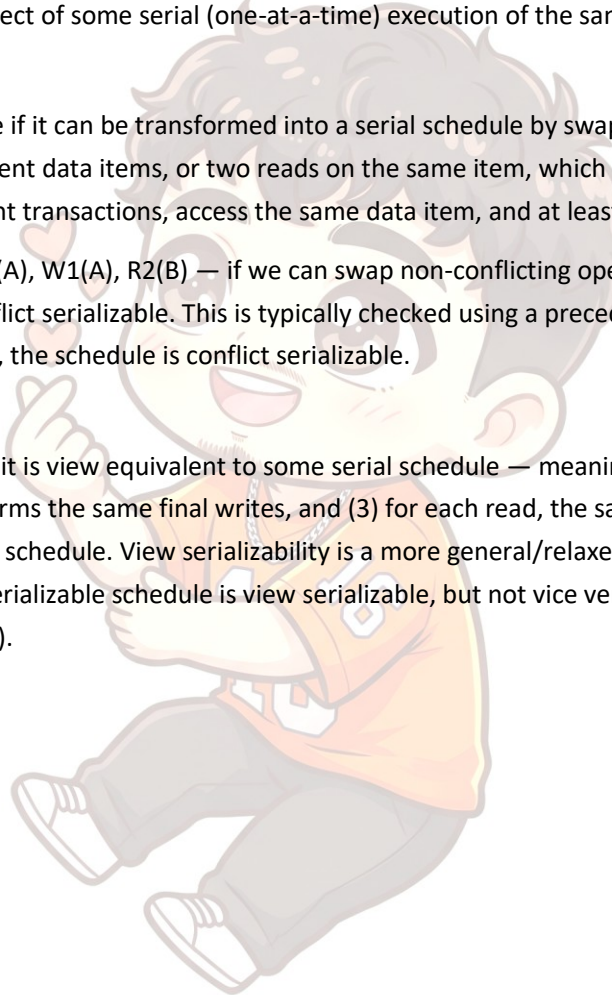
Conflict Serializability

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations (operations on different data items, or two reads on the same item, which don't conflict). Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a write.

Example: Schedule S: R1(A), W2(A), W1(A), R2(B) — if we can swap non-conflicting operations to get a serial order T1 then T2 (or T2 then T1), S is conflict serializable. This is typically checked using a precedence graph (serialization graph): if the graph has no cycle, the schedule is conflict serializable.

View Serializability

A schedule is view serializable if it is view equivalent to some serial schedule — meaning: (1) each transaction reads the same initial values, (2) performs the same final writes, and (3) for each read, the same transaction 'wrote' the value being read, as in the serial schedule. View serializability is a more general/relaxed condition than conflict serializability — every conflict serializable schedule is view serializable, but not vice versa (view serializability additionally allows 'blind writes').



Q3. What are transaction schedules? Explain serial schedule and non-serial schedule. [8 Marks]

In Simple English

A schedule is simply the order in which the individual steps (reads/writes) of one or more transactions actually get executed.

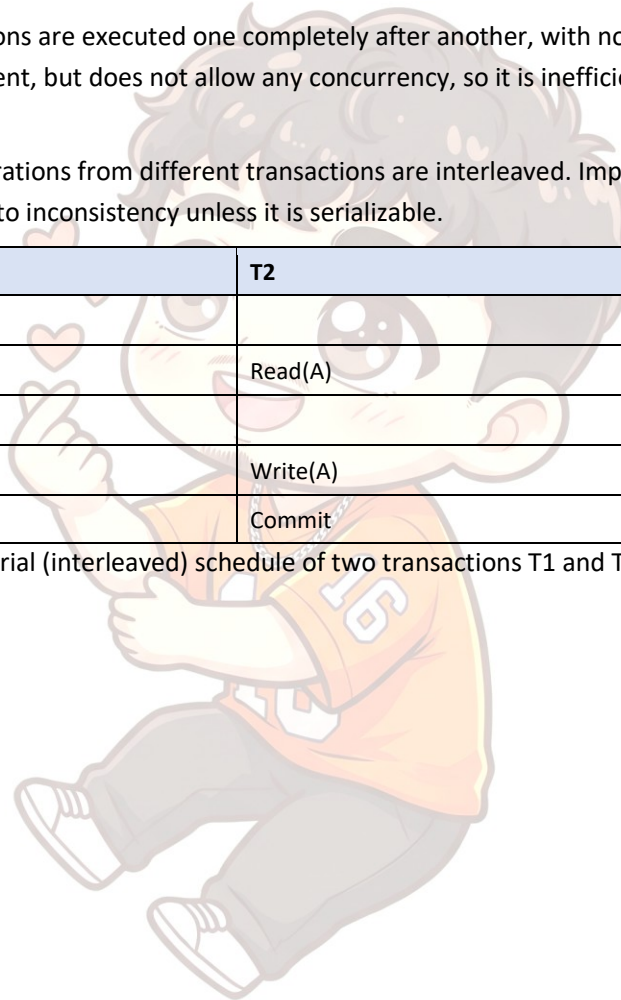
Technical Explanation

A schedule is a chronological sequence of operations (Read, Write, Commit, Abort) from one or more transactions, showing the order in which they are executed by the DBMS.

- **Serial Schedule:** Transactions are executed one completely after another, with no interleaving of operations. Simple and always consistent, but does not allow any concurrency, so it is inefficient (poor CPU/resource utilization).
- **Non-Serial Schedule:** Operations from different transactions are interleaved. Improves resource utilization and throughput, but may lead to inconsistency unless it is serializable.

T1	T2
Read(A)	
	Read(A)
Write(A)	
	Write(A)
Commit	Commit

The table above shows a non-serial (interleaved) schedule of two transactions T1 and T2 both operating on data item A.



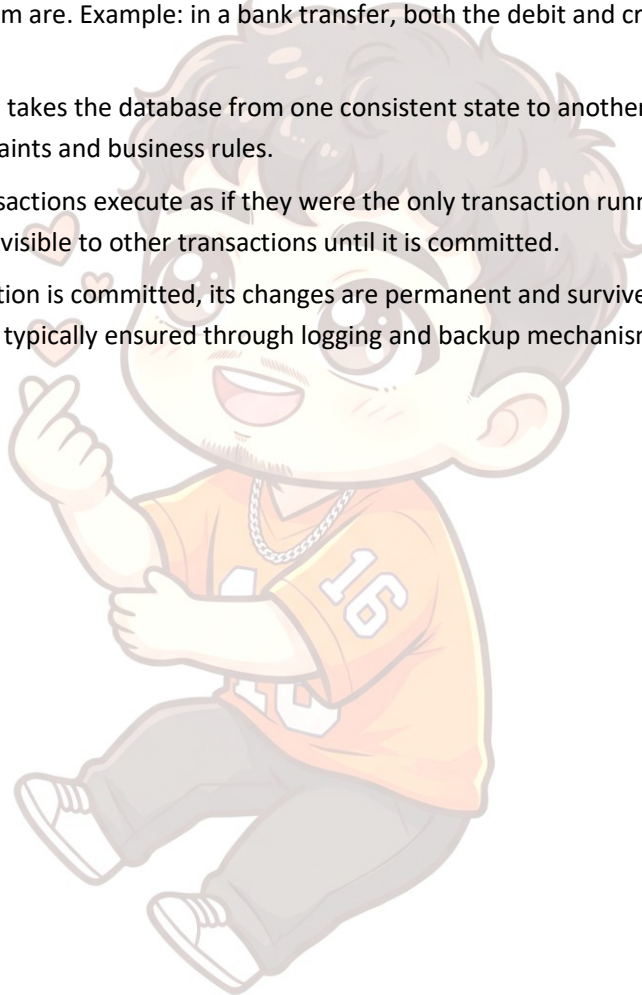
Q4. Write short notes on: ACID properties of transaction. [8 Marks]

In Simple English

ACID is a set of four promises a database makes about every transaction — that it will be done fully or not at all, that it keeps data correct, that it won't get mixed up with other transactions, and that once done, it stays done even if the power goes out.

Technical Explanation — The Four ACID Properties

- **Atomicity:** A transaction is treated as a single indivisible unit — either all of its operations are executed completely, or none of them are. Example: in a bank transfer, both the debit and credit must happen, or neither should.
- **Consistency:** A transaction takes the database from one consistent state to another consistent state, preserving all defined integrity constraints and business rules.
- **Isolation:** Concurrent transactions execute as if they were the only transaction running, so intermediate results of one transaction are not visible to other transactions until it is committed.
- **Durability:** Once a transaction is committed, its changes are permanent and survive any subsequent system failure (power loss, crash), typically ensured through logging and backup mechanisms.



8. Backup & Recovery System

Q1. What is checkpoint and transaction rollback? Explain log-based recovery with a suitable example. [8 Marks]

In Simple English

A checkpoint is like a save-point the database creates for itself periodically. A rollback means undoing a transaction's changes if something goes wrong. Log-based recovery means the DBMS keeps a diary (log) of everything that happens, so it can fix things after a crash by reading that diary.

Checkpoint

A checkpoint is a mechanism where the DBMS periodically saves the current state of the database (flushing all modified buffers/logs to disk) and records a <checkpoint> entry in the log. During recovery, the system only needs to consider transactions that started after the last checkpoint, drastically reducing the amount of log that must be scanned and redone/undone.

Transaction Rollback

Rollback (or abort) is the process of undoing all changes made by a transaction, restoring the database to the state it was in before the transaction began — used when a transaction cannot be completed successfully (e.g., due to a constraint violation, deadlock, or system error).

Log-Based Recovery

In log-based recovery, the DBMS maintains a log — a sequential record of every update, stored on stable storage, containing the transaction ID, data item, old value, and new value for every write operation. Two important recovery techniques based on logs are:

- **Deferred Update (Redo-only):** Actual database updates are deferred until a transaction commits; the log is used to redo committed transactions if a crash occurs before the update was physically written.
- **Immediate Update (Undo/Redo):** Updates may be written to the database before the transaction commits, so both undo (for uncommitted transactions) and redo (for committed transactions) may be required during recovery.

Example

```
<T1 start>
<T1, A, 500, 400>  -- old value 500, new value 400
<T1, B, 200, 300>
<T1 commit>
-- CRASH occurs here
```

On restart, the recovery manager scans the log: since <T1 commit> was recorded before the crash, the system redoes T1's writes (ensuring A=400 and B=300 are correctly applied to disk). If <T1 commit> had not appeared in the log, the system would instead undo the changes, restoring A=500 and B=200.

Q2. Write short notes on: Failure Classifications. [8 Marks]

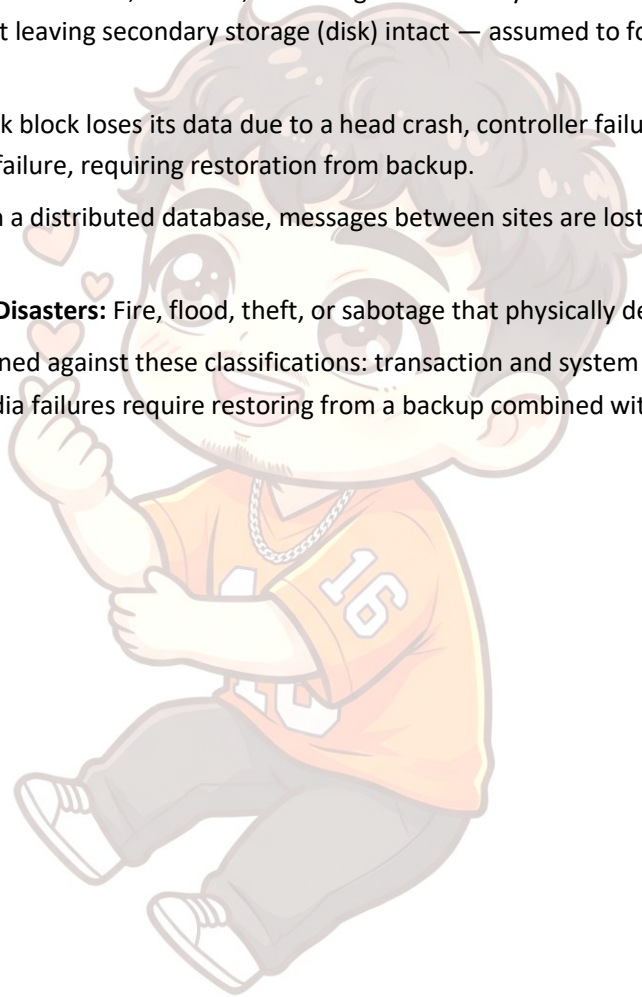
In Simple English

Things can go wrong with a database in different ways — sometimes it's just one transaction failing, sometimes the whole computer crashes, and sometimes even the disk itself gets physically damaged. Failure classification is just grouping these problems by type.

Technical Explanation — Types of Failure

- **Transaction Failure:** A single transaction cannot continue due to a logical error (e.g., bad input data, integrity constraint violation) or a system error (e.g., deadlock detected, resource unavailable).
- **System (Software) Crash:** A hardware, software, or OS bug causes the system to halt, losing the contents of volatile (main) memory but leaving secondary storage (disk) intact — assumed to follow the fail-stop assumption.
- **Disk / Media Failure:** A disk block loses its data due to a head crash, controller failure, or physical damage — typically the most serious failure, requiring restoration from backup.
- **Communication Failure:** In a distributed database, messages between sites are lost or a network link/site becomes unreachable.
- **Natural / Human-Caused Disasters:** Fire, flood, theft, or sabotage that physically destroys the storage media.

Recovery mechanisms are designed against these classifications: transaction and system failures are handled using logs and checkpoints, while media failures require restoring from a backup combined with the log (archival dump + roll-forward recovery).



9. Database Security

Q1. Differentiate between authentication and authorization. Discuss the importance of security and how encryption/decryption occurs. [8 Marks]

In Simple English

Authentication checks 'who are you?' — like showing an ID card at the door. Authorization checks 'what are you allowed to do once you're in?' — like being allowed only into certain rooms of a building.

Technical Explanation

Authentication: The process of verifying the identity of a user or system attempting to access the database, typically through a username/password, biometrics, or security tokens. It answers the question 'Is this really who they claim to be?'

Authorization: The process of granting or denying specific privileges (SELECT, INSERT, UPDATE, DELETE) to an authenticated user, determining what actions/data that user is permitted to access. It answers 'What is this verified user allowed to do?'

Aspect	Authentication	Authorization
Question answered	Who are you?	What are you allowed to do?
Occurs	Before authorization (first step)	After successful authentication
Mechanism	Username/password, OTP, biometrics	GRANT/REVOKE privileges, roles, access control lists
Example	Logging in to the database with valid credentials	A logged-in user being allowed to SELECT but not DELETE from a table

Importance of Database Security

- Protects sensitive/confidential data (e.g., personal, financial, medical records) from unauthorized access
- Prevents data breaches, theft, and malicious modification or destruction of data
- Ensures compliance with legal and regulatory requirements (e.g., data protection laws)
- Maintains the trust of users and the overall integrity/availability of the system

How Encryption / Decryption Occurs

Encryption is the process of converting readable data (plaintext) into an unreadable, coded form (ciphertext) using an encryption algorithm and a key, so that only authorized parties possessing the correct key can convert it back — a process called decryption.

- **Symmetric-Key (Private-Key) Encryption:** The same secret key is used for both encryption and decryption. It is fast and efficient for large volumes of data, but the key must be shared securely between sender and receiver beforehand. Example: DES, AES.
- **Asymmetric-Key (Public-Key) Encryption:** Uses a mathematically related key-pair — a public key (shared openly, used to encrypt) and a private key (kept secret, used to decrypt). Since the private key never needs to be shared, this solves the key-distribution problem of symmetric encryption, though it is computationally slower. Example: RSA.

In practice: the sender encrypts plaintext using the recipient's public key; only the recipient's corresponding private key can decrypt the ciphertext back into the original plaintext, ensuring confidentiality even if the ciphertext is intercepted during transmission.



10. Advanced Database Concepts & Storage

Q1. What are the advantages of ORM databases? Compare NoSQL databases with traditional relational databases. [8 Marks]

In Simple English

ORM lets programmers work with database tables as if they were normal objects/classes in their code, instead of writing raw SQL everywhere. NoSQL databases are a newer style of database that don't force data into rigid rows and columns, which makes them great for huge, fast-changing, or unstructured data.

ORM (Object-Relational Mapping) — Advantages

- Bridges the 'object-relational impedance mismatch' between object-oriented programming languages and relational tables
- Increases productivity — developers write in their programming language's objects instead of raw SQL
- Improves maintainability — schema and query logic are centralized in mapped classes
- Database independence — the same code can often work across different underlying relational databases (e.g., MySQL, PostgreSQL) with minimal change
- Built-in protection against common vulnerabilities like SQL injection through parameterized queries

NoSQL vs Traditional Relational Databases

Feature	Relational (SQL) Database	NoSQL Database
Data Model	Structured tables with fixed schema	Flexible — document, key-value, column-family, or graph
Schema	Rigid, must be defined in advance	Dynamic/schema-less, can evolve easily
Scalability	Vertical scaling (bigger server)	Horizontal scaling (add more servers) — built for distributed systems
Consistency	Strong (ACID) consistency	Often eventual consistency (BASE model)
Best For	Complex queries, transactions, structured data	Big data, real-time apps, unstructured/rapidly changing data
Examples	MySQL, Oracle, PostgreSQL	MongoDB, Cassandra, Redis, Neo4j

Q2. Define indexing. Explain index-sequential file organization with advantages and disadvantages. [8 Marks]

In Simple English

An index is like the index page at the back of a textbook — instead of reading every page to find a topic, you jump directly to the page number listed in the index. Database indexing works the same way to find data faster.

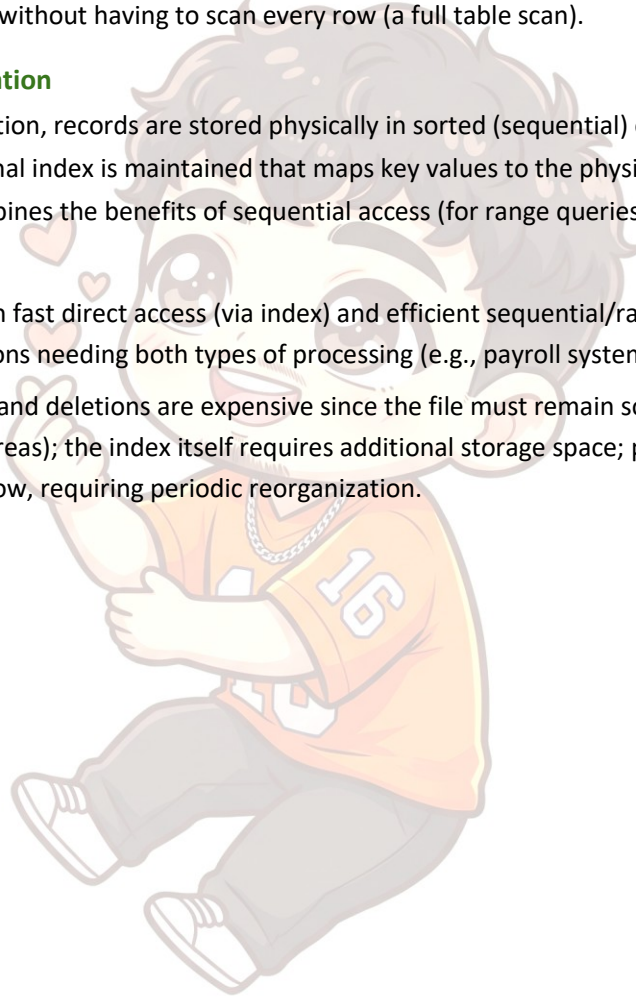
Technical Explanation — Indexing

Indexing is a data structure technique that provides fast, efficient access to rows in a database table, based on the values of one or more columns, without having to scan every row (a full table scan).

Index-Sequential File Organization

In index-sequential file organization, records are stored physically in sorted (sequential) order based on the key/search field, and an additional index is maintained that maps key values to the physical block address of the corresponding record. This combines the benefits of sequential access (for range queries) with an index for fast direct access.

- **Advantages:** Supports both fast direct access (via index) and efficient sequential/range access (since data is sorted); good for applications needing both types of processing (e.g., payroll systems).
- **Disadvantages:** Insertions and deletions are expensive since the file must remain sorted (may require reorganization or overflow areas); the index itself requires additional storage space; performance degrades over time as overflow chains grow, requiring periodic reorganization.



Q3. Explain the distributed database with its advantages and disadvantages over a centralized database. [8 Marks]

In Simple English

A centralized database keeps all data in one place. A distributed database spreads the data across multiple computers/locations that are connected via a network, but the system still makes it look like one single database to the user.

Technical Explanation

A distributed database is a collection of multiple, logically interrelated databases physically distributed across different sites on a computer network, managed by a Distributed DBMS (DDBMS) that makes the distribution transparent to the user.

Aspect	Centralized Database	Distributed Database
Location	Single location/server	Multiple sites/servers, geographically spread
Advantages	Simple to manage, easy to maintain consistency, lower complexity	Improved reliability/availability, faster local access, easy horizontal scalability, reflects organizational structure
Disadvantages	Single point of failure, can become a performance bottleneck, difficult to scale for geographically dispersed users	Complex to design and manage, higher cost, more difficult to maintain consistency/security across sites, increased software complexity

Q4. Write short notes on: Hash Index. [8 Marks]

In Simple English

A hash index converts a search key into a number using a formula (hash function), and that number directly tells the database exactly which 'bucket' the data is stored in — making lookups extremely fast.

Technical Explanation

A hash index applies a hash function $h(K)$ to a search key K to compute the address of the bucket where the corresponding record is stored. All records with keys that hash to the same value are stored in (or chained from) the same bucket.

- **Advantages:** Provides very fast $O(1)$ average-case lookup for exact-match (equality) queries.
- **Disadvantages:** Not efficient for range queries (e.g., Salary > 50000) since hashing destroys the ordering of keys; can suffer from bucket overflow if the hash function does not distribute keys uniformly.

Q5. Write short notes on: Stored Procedure. [8 Marks]

In Simple English

A stored procedure is a small program made of SQL statements that is saved inside the database itself, so it can be called and reused any time, just like calling a function in a programming language.

Technical Explanation

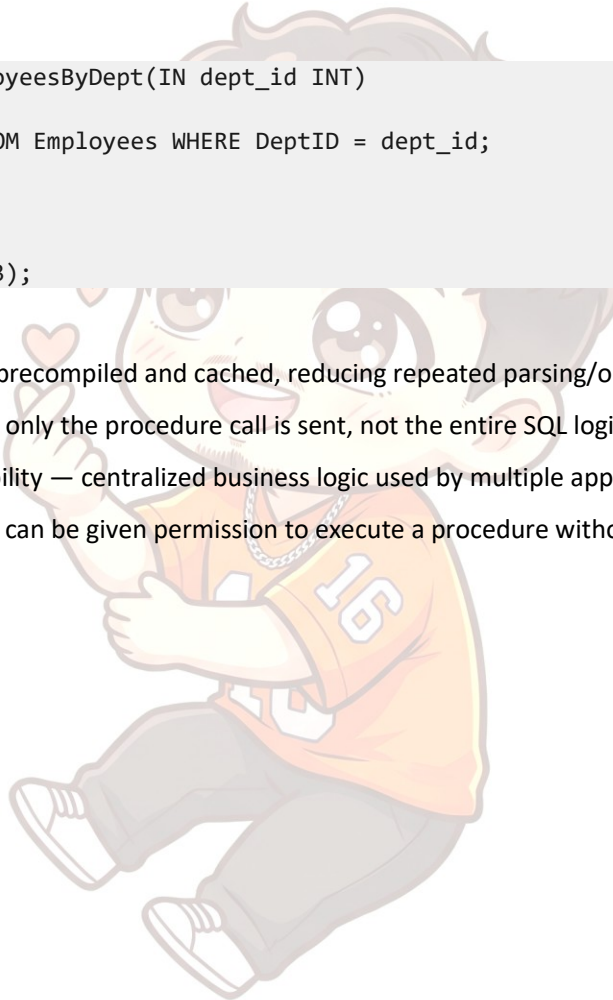
A stored procedure is a precompiled collection of one or more SQL statements, stored under a name in the database, which can accept input parameters, perform logic (loops, conditions), and return output values or result sets when called (executed).

```
CREATE PROCEDURE GetEmployeesByDept(IN dept_id INT)
BEGIN
    SELECT Name, Salary FROM Employees WHERE DeptID = dept_id;
END;

-- Calling the procedure
CALL GetEmployeesByDept(3);
```

Advantages

- Improved performance — precompiled and cached, reducing repeated parsing/optimization cost
- Reduced network traffic — only the procedure call is sent, not the entire SQL logic
- Reusability and maintainability — centralized business logic used by multiple applications
- Enhanced security — users can be given permission to execute a procedure without direct table access



Part B: University Question-Paper Specific Problems

This part gives complete, schema-exact answers to the numerical/query sub-questions from each specific past question paper, using the exact table names and columns given in each paper.

Pokhara University — 2025 Spring (New)

Q2(b). Relational Algebra Queries [8 Marks]

Schema: Student(SID, Name, Age, Dept), Course(CID, CName, Credits), Enroll(SID, CID, Grade)

(i) Names of students enrolled in the course "Database Systems"

```
 $\pi \text{ Name } ( \text{Student} \bowtie ( \sigma \text{ CName} = \text{'Database Systems'} ( \text{Course} ) \bowtie \text{Enroll} ) )$ 
```

We first select the row for 'Database Systems' in Course, join it with Enroll (on CID) to get the SIDs enrolled in it, then join with Student (on SID) and project the Name column.

(ii) Students (SID, Name) who are not enrolled in any course

```
 $\pi \text{ SID, Name } ( \text{Student} ) - \pi \text{ SID, Name } ( \text{Student} \bowtie \text{Enroll} )$ 
```

This uses set difference: all students, minus those who appear in at least one Enroll record.

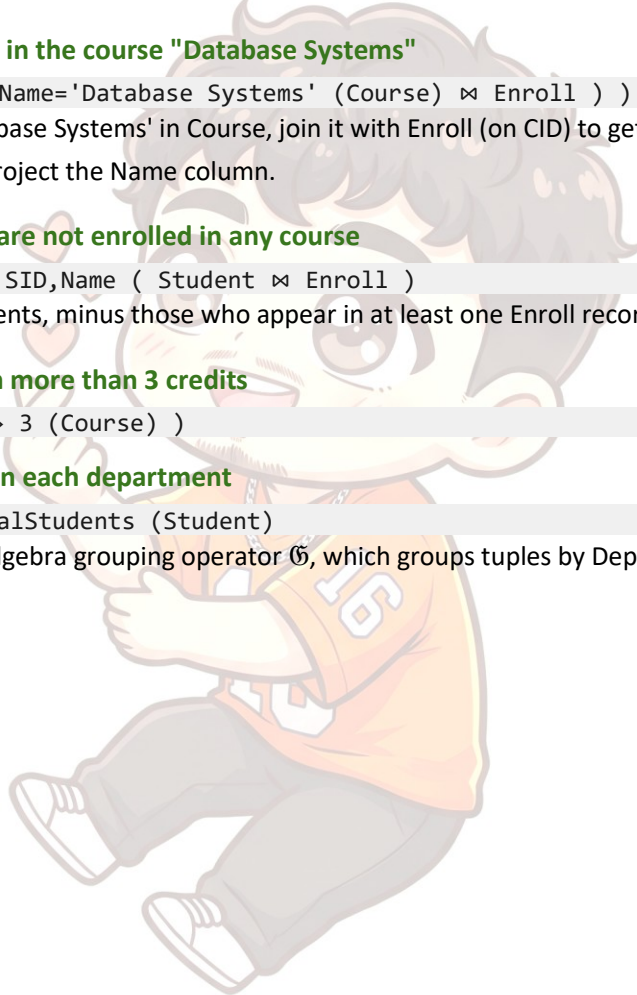
(iii) Courses (CID, CName) with more than 3 credits

```
 $\pi \text{ CID, CName } ( \sigma \text{ Credits} > 3 ( \text{Course} ) )$ 
```

(iv) Total number of students in each department

```
 $\text{Dept} \ \Join \ \text{COUNT}(\text{SID}) \ \text{AS TotalStudents } ( \text{Student} )$ 
```

(Using the extended relational algebra grouping operator $\Join$, which groups tuples by Dept and applies the aggregate function COUNT on SID.)



Q3(b). SQL Queries [8 Marks]

Schema: Employees(EmpID, Name, DeptID, Salary, JoinDate), Departments(DeptID, DeptName, Location), Projects(ProjectID, ProjectName, Budget, DeptID)

(i) Create the table Employees with appropriate constraints

```
CREATE TABLE Employees (  
    EmpID      INT PRIMARY KEY,  
    Name       VARCHAR(50) NOT NULL,  
    DeptID     INT,  
    Salary     DECIMAL(10,2) CHECK (Salary > 0),  
    JoinDate   DATE NOT NULL,  
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)  
);
```

(ii) Name, Salary of employees in the 'IT' department

```
SELECT E.Name, E.Salary  
FROM Employees E, Departments D  
WHERE E.DeptID = D.DeptID AND D.DeptName = 'IT';
```

(iii) Increase the salary of employees in the 'IT' department by 10%

```
UPDATE Employees  
SET Salary = Salary * 1.10  
WHERE DeptID = (SELECT DeptID FROM Departments WHERE DeptName = 'IT');
```

(iv) Delete all projects with a budget less than 40,000

```
DELETE FROM Projects WHERE Budget < 40000;
```

Q2(b). SQL Statements [8 Marks]

Schema: Students(student_id, name, dob, gender), Courses(course_id, course_name, credits),
Instructors(instructor_id, name, department), Enrollments(enrollment_id, student_id, course_id, instructor_id,
enrolled_date)

(i) List all courses a student named 'Sujan' is enrolled in

```
SELECT C.course_name
FROM Courses C, Enrollments E, Students S
WHERE C.course_id = E.course_id
      AND E.student_id = S.student_id
      AND S.name = 'Sujan';
```

(ii) Number of students enrolled on each course

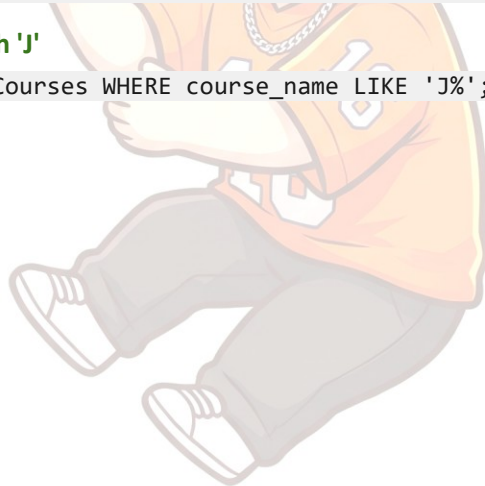
```
SELECT course_id, COUNT(student_id) AS TotalStudents
FROM Enrollments
GROUP BY course_id;
```

(iii) Average number of credits taken by students

```
SELECT AVG(TotalCredits) AS AvgCredits FROM (
  SELECT E.student_id, SUM(C.credits) AS TotalCredits
  FROM Enrollments E, Courses C
  WHERE E.course_id = C.course_id
  GROUP BY E.student_id
) AS StudentCredits;
```

(iv) Course names starting with 'J'

```
SELECT course_name FROM Courses WHERE course_name LIKE 'J%';
```



Q3(b). Normalization: 2NF → 3NF Table [8 Marks]

Given schema: (Name, Address, Phone, Salary, Post) with Name as Primary Key. This is the same normalization scenario worked through in detail in Section 5, Q2 — repeated here in the exact context of this paper's table:

Name	Address	Phone	Salary	Post
Ram	Pokhara	9800...	40000	Manager
Sita	Kathmandu	9810...	40000	Manager
Hari	Butwal	9820...	25000	Clerk

Transitive dependency present: Name → Post → Salary (Salary depends on Post, a non-key attribute, not directly on Name). This violates 3NF.

Anomalies (Yes, they occur)

- **Insertion Anomaly:** Cannot record the salary for a new Post (e.g., 'Accountant') until an employee with that post exists.
- **Update Anomaly:** If Manager's salary changes, every row with Post = 'Manager' must be updated — a missed row causes inconsistency.
- **Deletion Anomaly:** Deleting the only 'Clerk' employee erases the fact that Clerks earn 25000.

Decomposition to 3NF

Employee (Name, Address, Phone, Post)	-- PK: Name
PostSalary (Post, Salary)	-- PK: Post

This removes the transitive dependency: Salary now depends only on Post (its own primary key), eliminating all three anomalies.

Q5(a). How do encryption and decryption occur in private-key and public-key cryptography?

[8 Marks]

This concept is explained in full in Section 9, Q1 of this document (Database Security). In brief:

- **Private-Key (Symmetric) Cryptography:** The same secret key encrypts and decrypts the message. The sender encrypts plaintext with the shared secret key; the receiver uses the identical key to decrypt the ciphertext back to plaintext. Fast, but both parties must securely share the key in advance (e.g., AES, DES).
- **Public-Key (Asymmetric) Cryptography:** Each user has a key pair — a public key (shared openly) and a private key (kept secret). The sender encrypts the message using the receiver's public key; only the receiver's matching private key can decrypt it. This avoids the need to share any secret key beforehand (e.g., RSA).



Q2(a). Relational Algebra [8 Marks]

Schema: employee(eid, name, position, age), project(pid, name, duration, budget), works(eid, pid, location, salary)

(i) All 'Managers' who are above 30 years

```
 $\sigma_{\text{position}='Manager' \text{ AND } \text{age} > 30}(\text{employee})$ 
```

(ii) Project budget running for less than 45 days

```
 $\pi_{\text{name}, \text{budget}}(\sigma_{\text{duration} < 45}(\text{project}))$ 
```

(iii) Update the project name to 'Business Website' where pid is P098

Pure relational algebra is a query language, not an update language, so this is conceptually represented as a reassignment of the relation:

```
 $\text{project} \leftarrow (\sigma_{\text{pid} \neq 'P098'}(\text{project})) \cup \{(\text{pid}: 'P098', \text{name}: 'Business Website', \text{duration}, \text{budget})\}$ 
```

In practice (SQL) this update is written as:

```
UPDATE project SET name = 'Business Website' WHERE pid = 'P098';
```

(iv) Project locations where all the 'Software Developers' work

This is a classic Division ($\div$) query — we want locations L such that every Software Developer works at location L.

```
 $\pi_{\text{eid}, \text{location}}(\text{works}) \div \pi_{\text{eid}}(\sigma_{\text{position}='Software Developer'}(\text{employee}))$ 
```

Explanation: We first find the set of all Software Developer eids. Dividing the (eid, location) pairs of Works by this set of eids gives exactly the locations at which every single Software Developer is present.



Q2(a). Relational Algebraic Expressions [8 Marks]

Schema: Employee(Empname, street, city), Works(Empname, post, cmpname, Salary), Company(cmpname, location)

(i) An employee named Robert is promoted from Assistant Manager to Manager

Conceptually (RA update notation):

```
Works ← ( σ Empname≠'Robert' (Works) ) ∪ { (Empname:'Robert', post:'Manager', cmpname, Salary) }
```

Equivalent SQL:

```
UPDATE Works SET post = 'Manager' WHERE Empname = 'Robert' AND post = 'Assistant Manager';
```

(ii) Update Company relation so that all companies located in Dhangadhi are shifted to Kathmandu

```
Company ← ( σ location≠'Dhangadhi' (Company) ) ∪ ρ( π cmpname,location←'Kathmandu' ( σ location='Dhangadhi' (Company) ) )
```

Equivalent SQL:

```
UPDATE Company SET location = 'Kathmandu' WHERE location = 'Dhangadhi';
```

(iii) Remove all the records of employees who live in Butwal

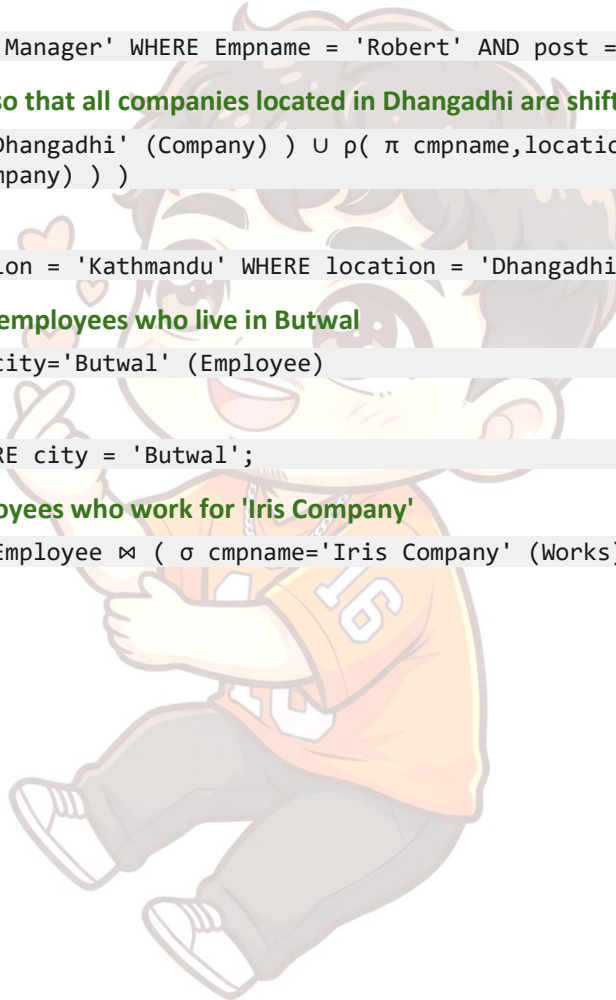
```
Employee ← Employee - σ city='Butwal' (Employee)
```

Equivalent SQL:

```
DELETE FROM Employee WHERE city = 'Butwal';
```

(iv) Name, street, city of employees who work for 'Iris Company'

```
π Empname,street,city ( Employee ⋈ ( σ cmpname='Iris Company' (Works) ) )
```



Q2(b). SQL Statements [8 Marks]

Schema: Customer(CustID, Name, Address, Phone), Orders(OrderID, CustID, OrderDate, TotalAmount), Product(ProdID, Name, Price), OrderDetails(OrderID, ProdID, Quantity)

(i) Create the Orders table with appropriate constraints

```
CREATE TABLE Orders (  
    OrderID      INT PRIMARY KEY,  
    CustID       INT NOT NULL,  
    OrderDate    DATE NOT NULL,  
    TotalAmount  DECIMAL(10,2) CHECK (TotalAmount >= 0),  
    FOREIGN KEY (CustID) REFERENCES Customer(CustID)  
);
```

(ii) Details of products that have never been ordered

```
SELECT * FROM Product  
WHERE ProdID NOT IN (SELECT ProdID FROM OrderDetails);
```

(iii) Change the price of a product named "iPhone" to 150000

```
UPDATE Product SET Price = 150000 WHERE Name = 'iPhone';
```

(iv) Names of customers who have placed orders worth more than 5000

```
SELECT DISTINCT C.Name  
FROM Customer C, Orders O  
WHERE C.CustID = O.CustID AND O.TotalAmount > 5000;
```

